Thinking in Objects

CSE 114: Introduction to Object-Oriented Programming

Contents

- Immutable Objects and Classes
- Scope of Variables and Default values
- The **`this`** Keyword
- Calling Overloaded Constructors
- Class Abstraction and Encapsulation
  - Designing and implementing the Loan Class
  - Designing and implementing the BMI Class
  - Designing and implementing the Course Class
  - Designing and implementing the StackOfIntegers Class
- The String Class in detail
  - Regular Expressions
  - Command-Line Parameters
- StringBuilder and StringBuffer
- The Character Class
- Designing Classes

Immutable Objects and Classes

- ***Immutable object***: the contents of an object cannot be changed once the object is created
  - its class is called an ***immutable class***
    - Example immutable class: no set method in the Circle class:

```
public class Circle{
    private double radius;
    public Circle() { }
    public Circle(double radius) {
        this.radius = radius;
    }
    public double getRadius() {
        return radius;
    }
}
```

    - radius is private and cannot be changed without a set method

What Class is Immutable?

- A class with all private data fields and without mutators is not necessarily immutable!
- An ***immutable class***:
  1. It must mark all data fields `private`!
  2. Provide no mutator (set) methods!
  3. Provide no accessor methods that would return a reference to a mutable data field object!

Example mutable

```
public class Student {
  private int id;
  private BirthDate birthDate;
  public Student(int ssn, int year,
         int month, int day) {
    id = ssn;
    birthDate =
         new BirthDate(year, month, day);
  }
  public int getId() {
    return id;
  }
  public BirthDate getBirthDate() {
    return birthDate;
  }
}
```

```
public class BirthDate {
  private int year;
  private int month;
  private int day;

  public BirthDate(int newYear,
      int newMonth, int newDay) {
    year = newYear;
    month = newMonth;
    day = newDay;
  }
  public void setYear(int newYear){
    year = newYear;
  }
  public int getYear() {
    return year;
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    Student student = new Student(111223333, 1998, 1, 1);
    student.getBirthDate().setYear(2050);
    // Now the student birth year is changed:
    System.out.println(student.getBirthDate().getYear()); // 2050
  }
}
```

Scope of Variables and Default values

- The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable
  - Also a local variable must be **initialized** explicitly before it can be used.

vs.

- Data Field Variables can be declared anywhere inside a class
  - The scope of instance and static variables is the entire class!
  - Initialized with default values.

The `this` Keyword

- The **`this`** keyword is the name of a reference that refers to an object itself
- Common uses of the **`this`** keyword:
  1. Reference a class's "***hidden***" *data fields*.
  2. To enable a constructor to **invoke another constructor** of the same class as the first statement in the constructor.

Reference the Hidden Data Fields

```java
public class Foo {
  private int i = 5;
  private static double k = 0;

  void setI(int i) {
    this.i = i;
  }

  static void setK(double k) {
    Foo.k = k;
  }
}
```

```
Suppose that f1 and f2 are two objects of Foo.

Invoking f1.setI(10) is to execute
   this.i = 10, where this refers f1

Invoking f2.setI(45) is to execute
   this.i = 45, where this refers f2
```

Calling Overloaded Constructors

```
public class Circle {
  private double radius;

  public Circle(double radius) {
    this.radius = radius;
  }

  public Circle() {
    this(1.0);
  }

  public double getArea() {
    return this.radius * this.radius * Math.PI;
  }
}
```

this must be explicitly used to reference the data field radius of the object being constructed

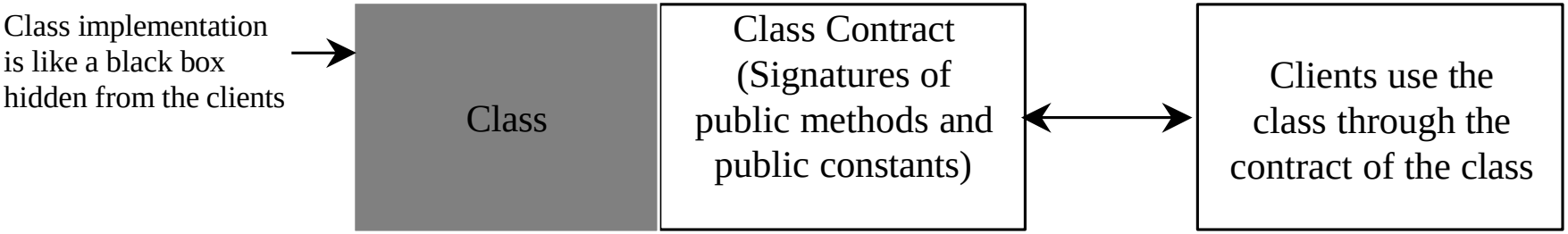
this is used to invoke another constructor

Every instance variable belongs to an instance represented by this, which is normally omitted

Class Abstraction and Encapsulation

Class Abstraction = separate class implementation from the use of the class (API)

- The creator of the class provides a description of the class and let the user know how the class can be used.
- The user does not need to know how the class is implemented: it is ***encapsulated*** (private fields and only the public API is used).



Designing the Loan Class

Loan	
-annualInterestRate: double	The annual interest rate of the loan (default: 2.5).
-numberOfYears: int	The number of years for the loan (default: 1)
-loanAmount: double	The loan amount (default: 1000).
-loanDate: Date	The date this loan was created.
+Loan()	Constructs a default Loan object.
+Loan(annualInterestRate: double, numberOfYears: int, loanAmount: double)	Constructs a loan with specified interest rate, years, and loan amount.
+getAnnualInterestRate(): double	Returns the annual interest rate of this loan.
+getNumberOfYears(): int	Returns the number of the years of this loan.
+getLoanAmount(): double	Returns the amount of this loan.
+getLoanDate(): Date	Returns the date of the creation of this loan.
+setAnnualInterestRate(annualInterestRate: double): void	Sets a new annual interest rate to this loan.
+setNumberOfYears(numberOfYears: int): void	Sets a new number of years to this loan.
+setLoanAmount(loanAmount: double): void	Sets a new amount to this loan.
+getMonthlyPayment(): double	Returns the monthly payment of this loan.
+getTotalPayment(): double	Returns the total payment of this loan.

```java
public class Loan {
  private double annualInterestRate;
  private int numberOfYears;
  private double loanAmount;
  private java.util.Date loanDate;
  public Loan() {
    this(2.5, 1, 1000);
  }
  public Loan(double annualInterestRate, int numberOfYears,
      double loanAmount) {
    this.annualInterestRate = annualInterestRate;
    this.numberOfYears = numberOfYears;
    this.loanAmount = loanAmount;
    loanDate = new java.util.Date();
  }
  public double getMonthlyPayment() {
    double monthlyInterestRate = annualInterestRate / 1200;
    double monthlyPayment = loanAmount * monthlyInterestRate / (1 -
      (Math.pow(1 / (1 + monthlyInterestRate), numberOfYears * 12)));
    return monthlyPayment;
  }
  public double getTotalPayment() {
    double totalPayment = getMonthlyPayment() * numberOfYears * 12;
    return totalPayment;
  } ...
}
```

The BMI Class

The get methods for these data fields are provided in the class, but omitted in the UML diagram for brevity.

BMI	
-name: String	The name of the person.
-age: int	The age of the person.
-weight: double	The weight of the person in pounds.
-height: double	The height of the person in inches.
+BMI(name: String, age: int, weight: double, height: double)	Creates a BMI object with the specified name, age, weight, and height.
+BMI(name: String, weight: double, height: double)	Creates a BMI object with the specified name, weight, height, and a default age 20.
+getBMI(): double	Returns the BMI
+getStatus(): String	Returns the BMI status (e.g., normal, overweight, etc.)

```java
public class BMI {
  private String name;
  private int age;
  private double weight; // in pounds
  private double height; // in inches
  public static final double KILOGRAMS_PER_POUND = 0.45359237;
  public static final double METERS_PER_INCH = 0.0254;
  public BMI(String name, int age, double weight, double height) {
 this.name = name; this.age = age; this.weight = weight; this.height = height;
  }
  public double getBMI() {
    double bmi = weight * KILOGRAMS_PER_POUND /
     ((height * METERS_PER_INCH) * (height * METERS_PER_INCH));
    return Math.round(bmi * 100) / 100.0;
  }
  public String getStatus() {
    double bmi = getBMI();
    if (bmi < 16) return "seriously underweight";
    else if (bmi < 18) return "underweight";
    else if (bmi < 24) return "normal weight";
    else if (bmi < 29) return "over weight";
    else if (bmi < 35) return "seriously over weight";
    else return "gravely over weight";
  }
  public String getName() {return name; }
  public int getAge() {    return age;  }
  public double getWeight() {    return weight;  }
  public double getHeight() {    return height;  }
}
```

Example: The Course Class

Course	
-name: String	The name of the course.
-students: String[]	The students who take the course.
-numberOfStudents: int	The number of students (default: 0).
+Course(name: String)	Creates a Course with the specified name.
+getName(): String	Returns the course name.
+addStudent(student: String): void	Adds a new student to the course list.
+getStudents(): String[]	Returns the students for the course.
+getNumberOfStudents(): int	Returns the number of students for the course.

```java
public class Course {
  private String courseName;
  private String[] students = new String[100];
  private int numberOfStudents;
  public Course(String courseName) {
    this.courseName = courseName;
  }
  public void addStudent(String student) {
    if(numberOfStudents >= students.length){
       String[] temp = new String[students.length * 2];
       System.arrayCopy(students, 0, temp, 0, students.length);
       students = temp;
    }
    students[numberOfStudents++] = student;
  }
  public String[] getStudents() {
    return students;
  }
  public int getNumberOfStudents() {
    return numberOfStudents;
  }
  public String getCourseName() {
    return courseName;
  }
}
```